

CSC240_LinkedQueue_Practice/CSC240LinkedQueueCircularTestDriver

```
1  //=====
2  // Name      : CSC240_Queue_Linked.cpp
3  // Author     : Kazim Zaidi
4  // Version    : 9/23/2026
5  // Copyright  : I'll fucking shoot you
6  // Description: Make this bitch circular
7  //=====
8  // Header file for Queue ADT.
9  // Template version of the Linked Queue using a front and rear pointer.
10 // #include "QueLinkedCircular.h"
11
12 #include <iostream>
13 #include <new>
14 #include <iostream>
15 #include <cstdint>
16
17 using namespace std;
18
19 template <class ItemType>
20
21 struct NodeType
22 {
23     ItemType info;
24     NodeType<ItemType> *next;
25 };
26
27 class FullQueue
28 {
29 };
30 class EmptyQueue
31 {
32 };
33
34 template <class ItemType>
35 class QueType
36 {
37 public:
38     QueType();
39     // Class constructor.
40     // Because there is a default constructor, the precondition
41     // that the queue has been initialized is omitted.
42     QueType(int max);
43     // Parameterized class constructor.
44     ~QueType();
45     // Class destructor.
46     QueType(const QueType &anotherQue);
47     // Copy constructor
```

```
48     void MakeEmpty();
49     // Function: Initializes the queue to an empty state.
50     // Post: Queue is empty.
51     bool IsEmpty() const;
52     // Function: Determines whether the queue is empty.
53     // Post: Function value = (queue is empty)
54     bool IsFull() const;
55     // Function: Determines whether the queue is full.
56     // Post: Function value = (queue is full)
57     void Enqueue(ItemType newItem);
58     // Function: Adds newItem to the rear of the queue.
59     // Post: If (queue is full) FullQueue exception is thrown
60     //         else newItem is at rear of queue.
61     void Dequeue(ItemType &item);
62     // Function: Removes front item from the queue and returns it in item.
63     // Post: If (queue is empty) EmptyQueue exception is thrown
64     //         and item is undefined
65     //         else front element has been removed from queue and
66     //         item is a copy of removed element.
67     void Print();
68     // Function: Display the contents of the QueType in the console output.
69     // Post: The QueType remains unchanged.
70
71 private:
72     //     NodeType<ItemType> *front;
73     NodeType<ItemType> *rear;
74 };
75
76 template <class ItemType>
77 void QueType<ItemType>::Print() // rewrote to circular
78 {
79     if (isEmpty())
80     {
81         throw EmptyQueue;
82     }
83     else
84     {
85         NodeType<ItemType> *tempPtr = rear;
86         tempPtr->next;
87         std::cout << "Front:";
88         while (tempPtr != rear)
89         {
90             std::cout << temp->info << " ";
91             tempPtr = tempPtr->next;
92         }
93         std::cout << tempPtr->info << ":Rear" << endl;
94     }
95 }
96
```

```
97  template <class ItemType>
98  QueType<ItemType>::QueType() // Class constructor. // rewrote to circular
99  // Post: front and rear are set to NULL.
100 {
101     // front = nullptr;
102     rear = nullptr;
103 }
104
105 template <class ItemType>
106 void QueType<ItemType>::MakeEmpty() // rewrote to circular
107 // Post: Queue is empty; all elements have been deallocated.
108 {
109     NodeType<ItemType> *tempPtr;
110
111     tempPtr = rear->next; // tempPtr pointing at first node
112
113     rear->next = nullptr;
114
115     while (tempPtr->next != nullptr)
116     {
117         rear = tempPtr;
118         tempPtr = tempPtr->next;
119         delete rear;
120     }
121     delete tempPtr; // tempPtr is nullptr -> reads the address after the
    disconnected rear node
122     rear = nullptr;
123 }
124
125 // Class destructor.
126 template <class ItemType>
127 QueType<ItemType>::~~QueType()
128 {
129     MakeEmpty();
130 }
131
132 template <class ItemType>
133 bool QueType<ItemType>::IsFull() const
134 // Returns true if there is no room for another ItemType
135 // on the free store; false otherwise.
136 {
137     NodeType<ItemType> *location;
138     try
139     {
140         location = new NodeType<ItemType>;
141         delete location;
142         return false;
143     }
144     catch (std::bad_alloc &)
```

```
145     {
146         return true;
147     }
148 }
149
150 template <class ItemType>
151 bool QueType<ItemType>::IsEmpty() const
152 // Returns true if there are no elements on the queue; false otherwise.
153 {
154     return (rear == nullptr);
155 }
156
157 template <class ItemType>
158 void QueType<ItemType>::Enqueue(ItemType newItem) // rewrote to circular
159 // Adds newItem to the rear of the queue.
160 // Pre: Queue has been initialized.
161 // Post: If (queue is not full) newItem is at the rear of the queue;
162 //       otherwise a FullQueue exception is thrown.
163
164 {
165     if (IsFull())
166         throw FullQueue();
167     else
168     {
169         NodeType<ItemType> newNode;
170         newNode = new NodeType<ItemType>; // pointer to newNode
171         newNode->info = newItem;
172
173         if (IsEmpty())
174         {
175             rear = newNode;
176             newNode->next = rear;
177         }
178         if (!IsEmpty())
179         {
180             newNode->next = rear->next;
181             rear->next = newNode;
182             rear = newNode;
183         }
184         // rear = newNode;
185     }
186 }
187
188 template <class ItemType>
189 void QueType<ItemType>::Dequeue(ItemType &item)
190 // Removes front item from the queue and returns it in item.
191 // Pre: Queue has been initialized and is not empty.
192 // Post: If (queue is not empty) the front of the queue has been
193 //       removed and a copy returned in item;
```

```
194 //      othersiwe a EmptyQueue exception has been thrown.
195 {
196     if (IsEmpty())
197         throw EmptyQueue();
198     else
199     {
200
201         NodeType<ItemType> *tempPtr;
202
203         tempPtr = rear->next;          // tempPtr points to first element
204         rear->next = rear->next->next; // last element points to second element
205         delete tempPtr;
206     }
207 }
208
209 template <class ItemType>
210 QueType<ItemType>::QueType(const QueType &anotherQue)
211 {
212
213     NodeType<ItemType> *ptr1;
214     NodeType<Itemtype> *ptr2;
215
216     if (anotherQue.rear == nullptr)
217     {
218         rear == nullptr;
219     }
220     ptr1 = anotherQue->rear;
221     ptr2 = new NodeType<ItemType>;
222
223     else
224     {
225         ptr1 = rear->next;
226
227         while (ptr1->next != rear)
228         {
229             ptr2 = new NodeType<ItemType>;
230             ptr2->info = ptr1
231         }
232     }
233
234     if (anotherQue.front == nullptr)
235         front = nullptr;
236     else
237     {
238         front = new NodeType<ItemType>;
239         front->info = anotherQue.front->info;
240         ptr1 = anotherQue.front->next;
241         ptr2 = front;
242         while (ptr1 != nullptr)
```

```
243     {
244         ptr2->next = new NodeType<ItemType>;
245         ptr2 = ptr2->next;
246         ptr2->info = ptr1->info;
247         ptr1 = ptr1->next;
248     }
249     ptr2->next = NULL;
250     rear = ptr2;
251 }
252 }
253
254 int main()
255 {
256     QueType<int> q;
257     q.Enqueue(8);
258     q.Enqueue(6);
259     q.Enqueue(7);
260     q.Enqueue(5);
261     q.Enqueue(3);
262     q.Enqueue(0);
263     q.Enqueue(9);
264     q.Print();
265     int numFront;
266     q.Dequeue(numFront);
267     cout << numFront << endl;
268     q.Print();
269     return 0;
270 }
271
```